

Introduction to Machine Learning

Richard Röttger

BMB547

2024

Table of Contents

- Introduction
- Machine Learning
- Logistic Regression

➤ The slides are based on https://evol.bio.lmu.de/_statgen/Rcourse/ws1617/

Table of Contents

- **Introduction**
- Machine Learning
- Logistic Regression

➤ The slides are based on https://evol.bio.lmu.de/_statgen/Rcourse/ws1617/

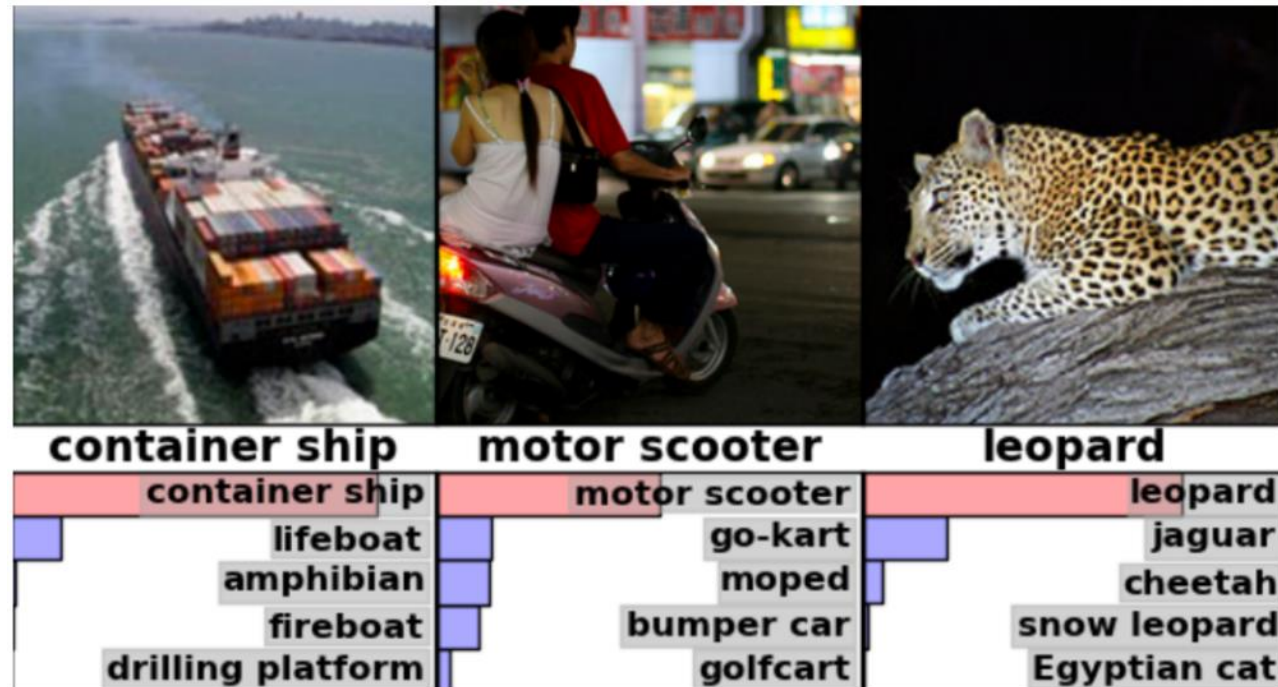
Kinds of Tasks solved using ML

- Classification
- Regression
- Transcription
- Machine Translation
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...

Kinds of Tasks solved using ML

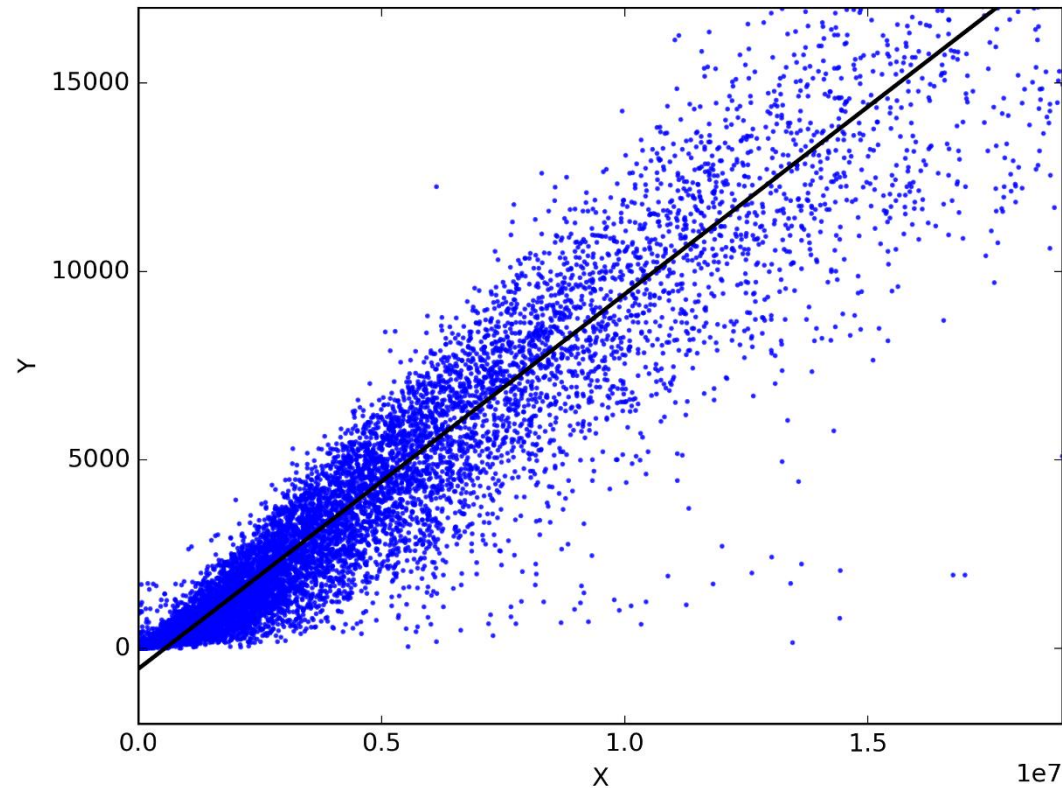
➤ Classification

- Regression
- Transcription
- Machine Translation
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...



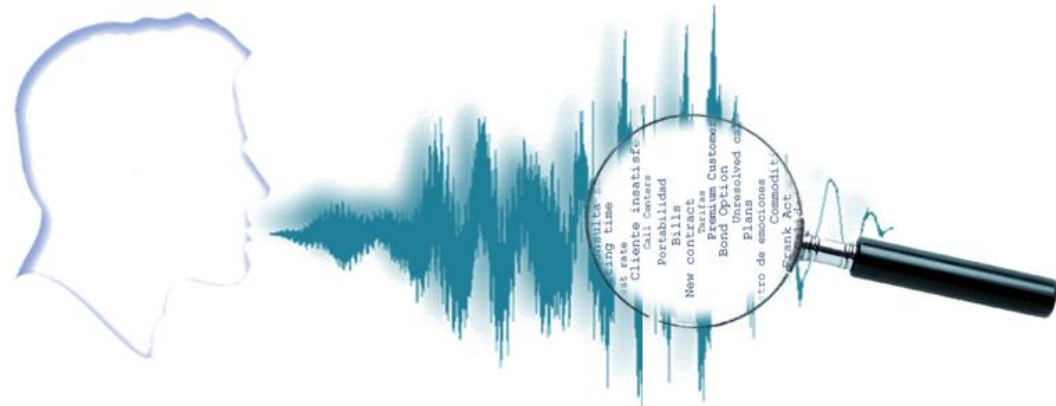
Kinds of Tasks solved using ML

- Classification
- **Regression**
- Transcription
- Machine Translation
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...



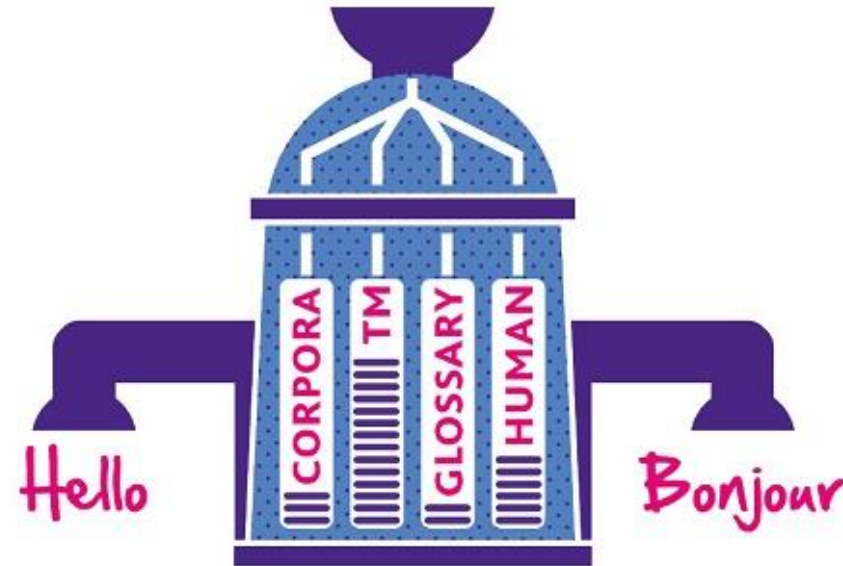
Kinds of Tasks solved using ML

- Classification
- Regression
- **Transcription**
- Machine Translation
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...



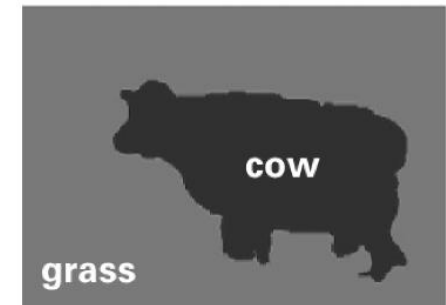
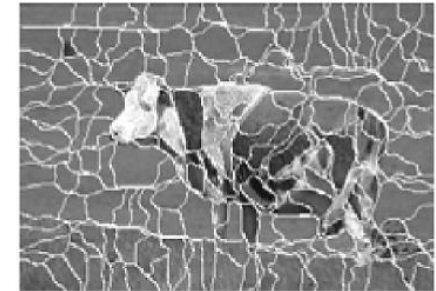
Kinds of Tasks solved using ML

- Classification
- Regression
- Transcription
- **Machine Translation**
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...



Kinds of Tasks solved using ML

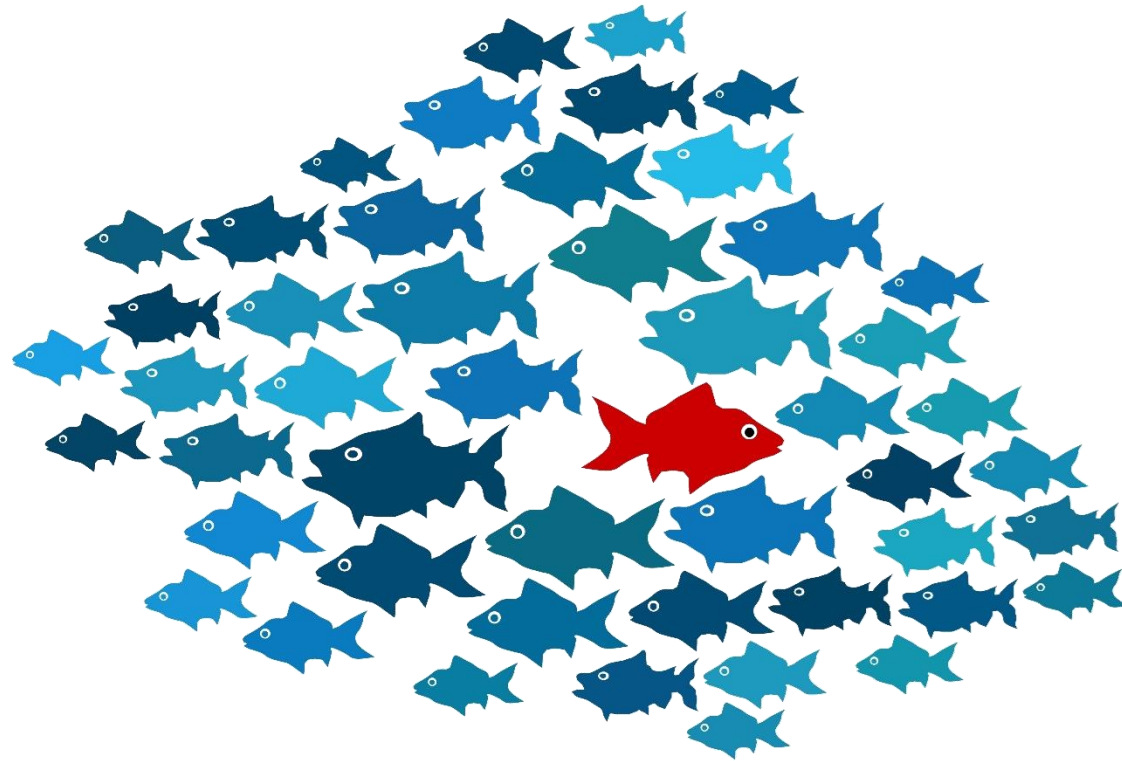
- Classification
- Regression
- Transcription
- Machine Translation
- **Structured Output**
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...



The man at bat readies to swing at the pitch while the umpire looks on.

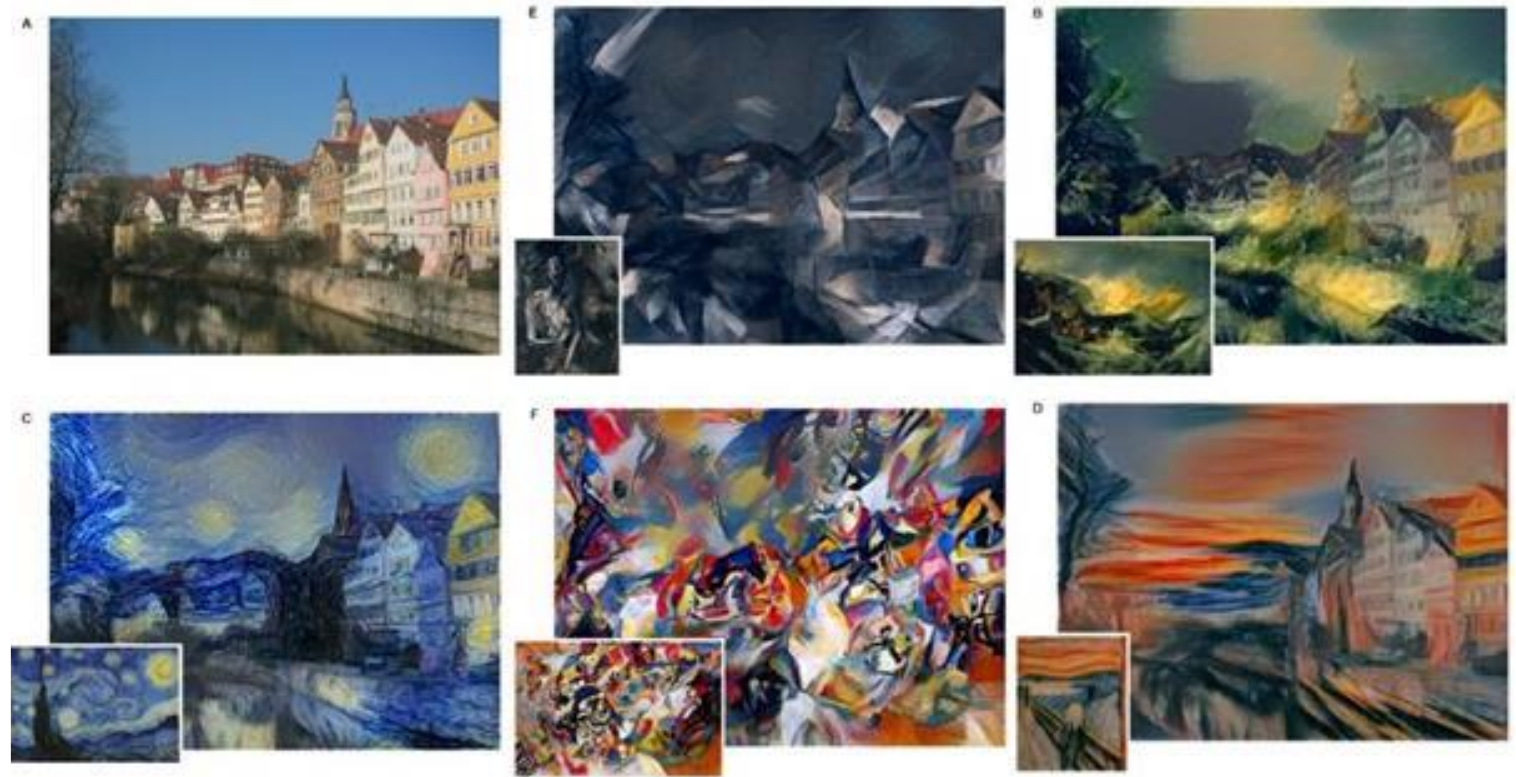
Kinds of Tasks solved using ML

- Classification
- Regression
- Transcription
- Machine Translation
- Structured Output
- **Anomaly Detection**
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...



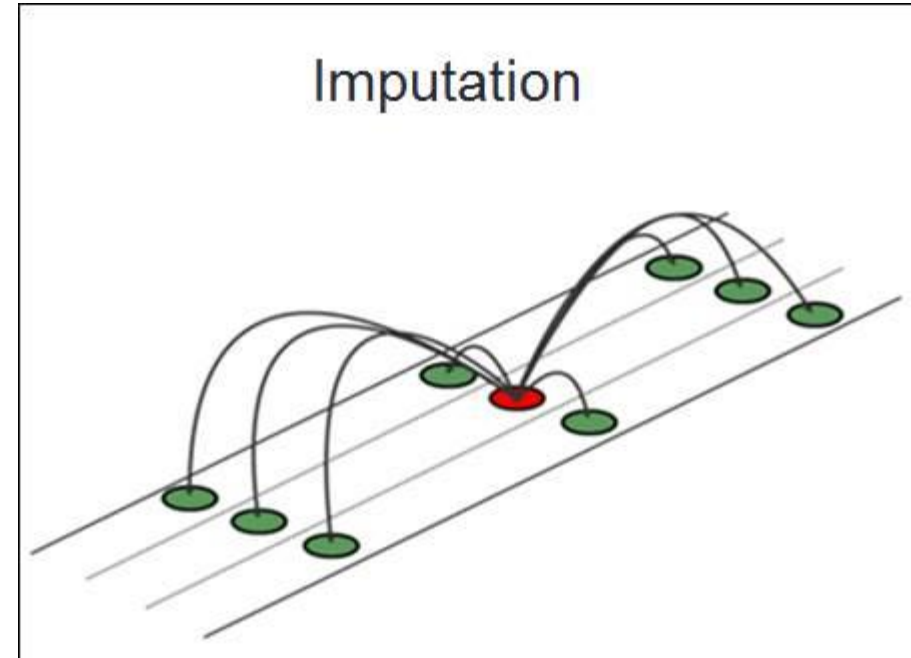
Kinds of Tasks solved using ML

- Classification
- Regression
- Transcription
- Machine Translation
- Structured Output
- Anomaly Detection
- **Synthesis and Sampling**
- Imputation of Missing Values
- Denoising
- Density Estimation
- ...



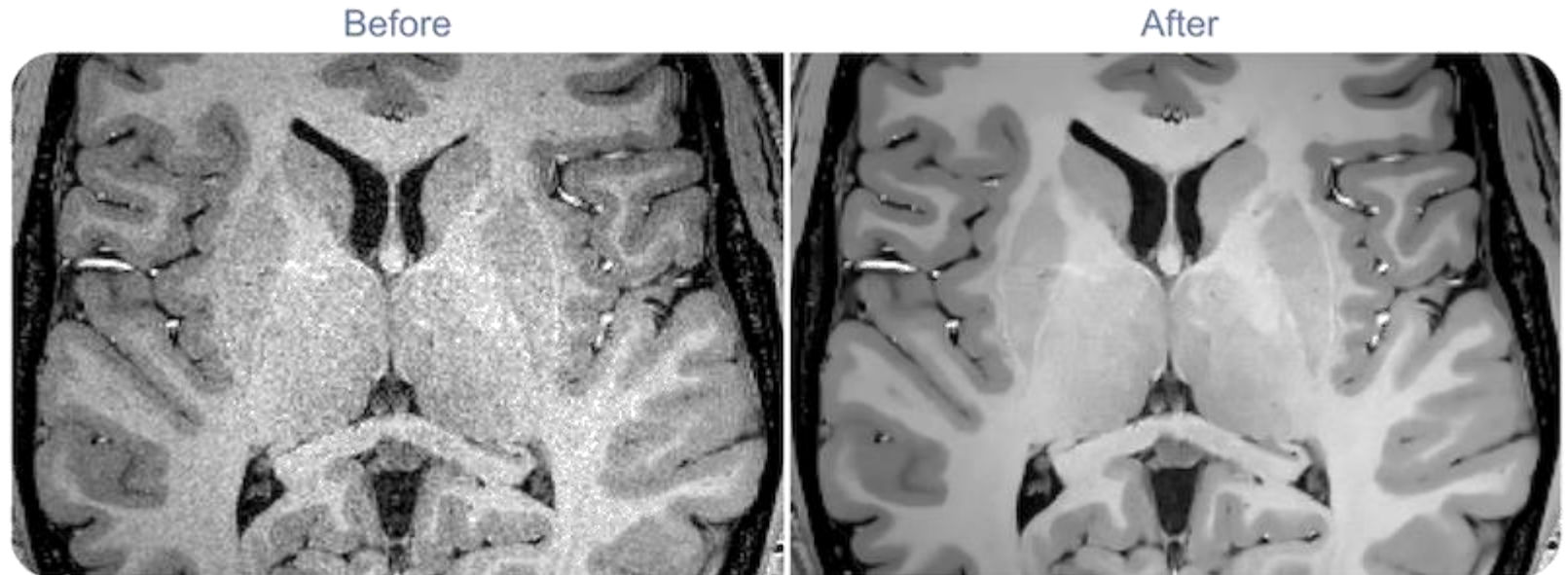
Kinds of Tasks solved using ML

- Classification
- Regression
- Transcription
- Machine Translation
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- **Imputation of Missing Values**
- Denoising
- Density Estimation
- ...



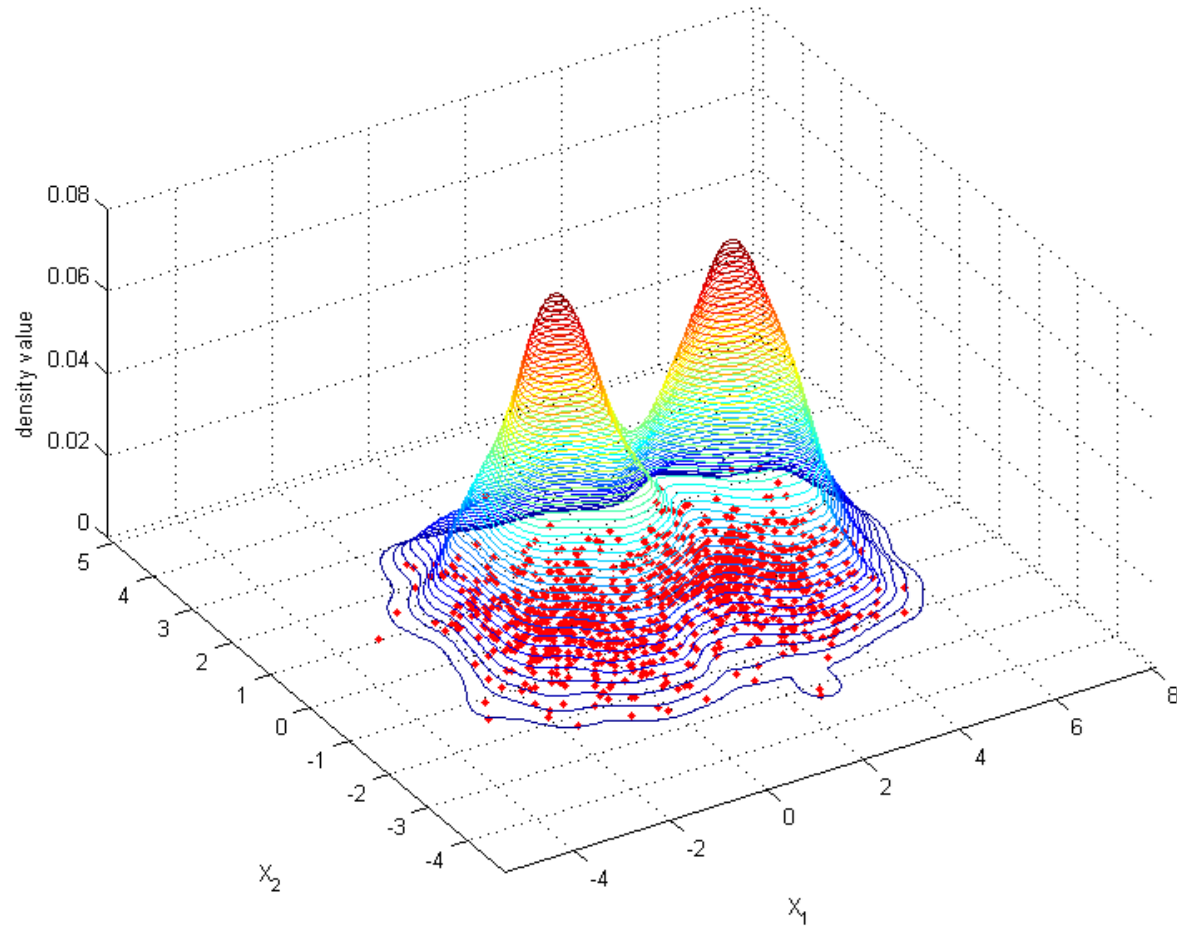
Kinds of Tasks solved using ML

- Classification
- Regression
- Transcription
- Machine Translation
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- **Denoising**
- Density Estimation
- ...



Kinds of Tasks solved using ML

- Classification
- Regression
- Transcription
- Machine Translation
- Structured Output
- Anomaly Detection
- Synthesis and Sampling
- Imputation of Missing Values
- Denoising
- **Density Estimation**
- ...



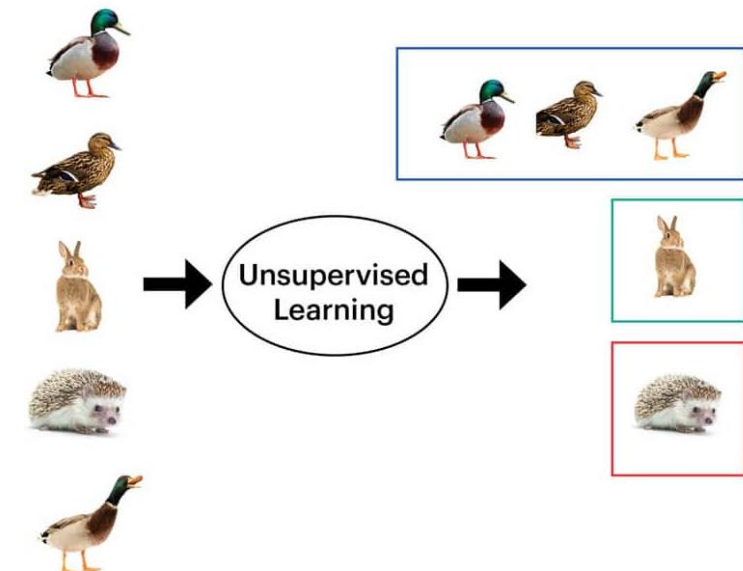
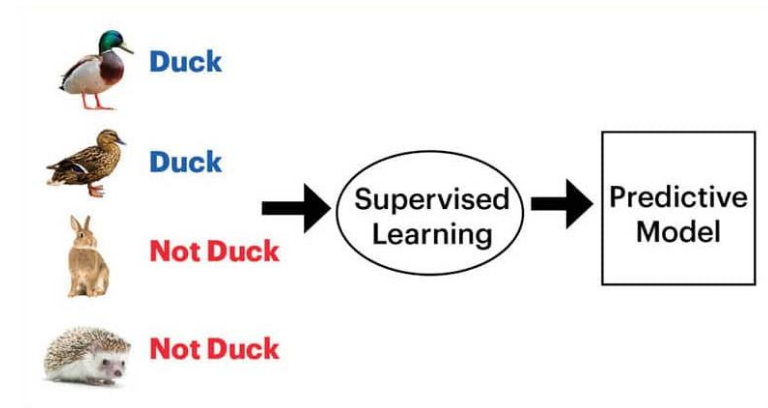
Types of Machine Learning

➤ Supervised Machine Learning

- Data is given together with the desired labels/outcome
- The machine learning task is to correctly predict the outcome
- Examples are Classification / Regression

➤ Unsupervised Machine Learning

- Used to analyze data
- Detect patterns / clusters in data
- Data is given without labels



What is Learning

- Now let's turn to the fundamentals. What is learning?
- **A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at task T , as measured by P , improves with experience E**

What is Learning

- Now let's turn to the fundamentals. What is learning?
- A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at task T , as measured by P , improves with experience E
- The Task T :
 - The task can have a multitude of different forms:
 - Classification
 - Prediction
 - Or even to drive a car or to beat the living crap out of GO players

What is Learning

- Now let's turn to the fundamentals. What is learning?
- **A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at task T , as measured by P , improves with experience E**
- **Performance P :**
 - For example:
 - Number of misclassifications
 - Number of won games
 - Number of accidents or rule infringements while driving the car

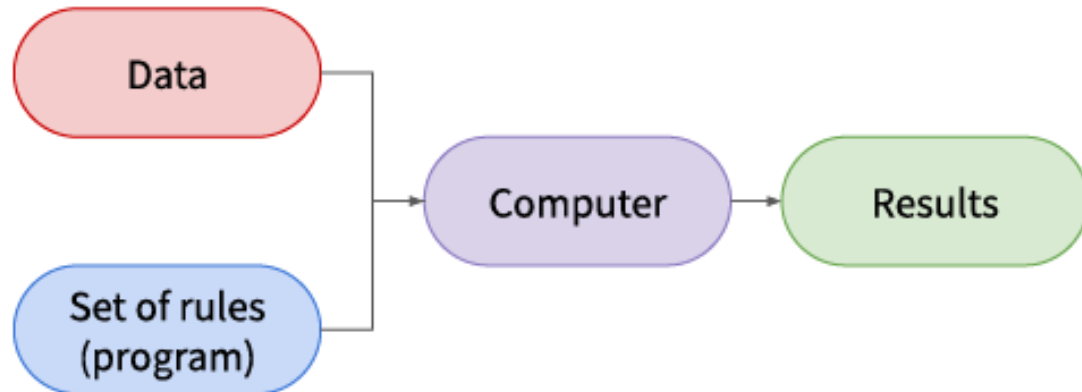
What is Learning

- Now let's turn to the fundamentals. What is learning?
- A computer program is said to learn from **experience E** with respect to some class of tasks **T** and performance measure **P**, if its performance at task **T**, as measured by **P**, improves with **experience E**
- **Experience E:**
 - Just means, the program should improve with an increasing number of examples

What does this mean for the Computer

- Ultimately we have to create a computer program that can learn with experience

Traditional programming



Machine learning

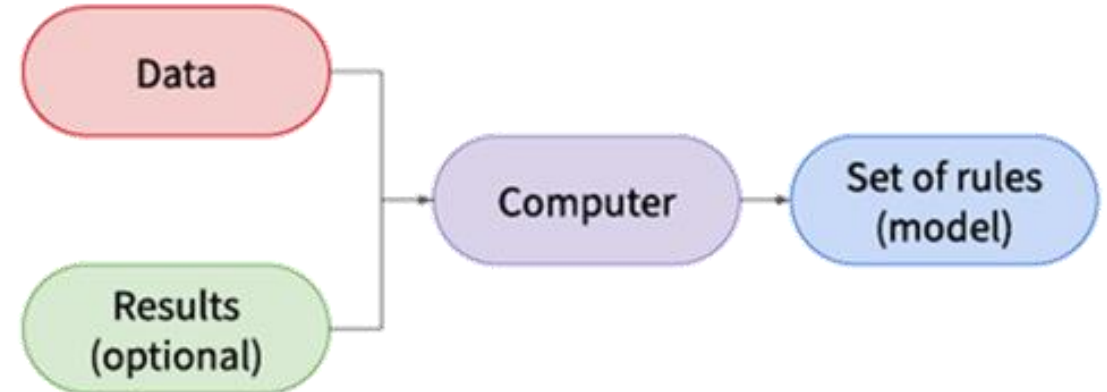


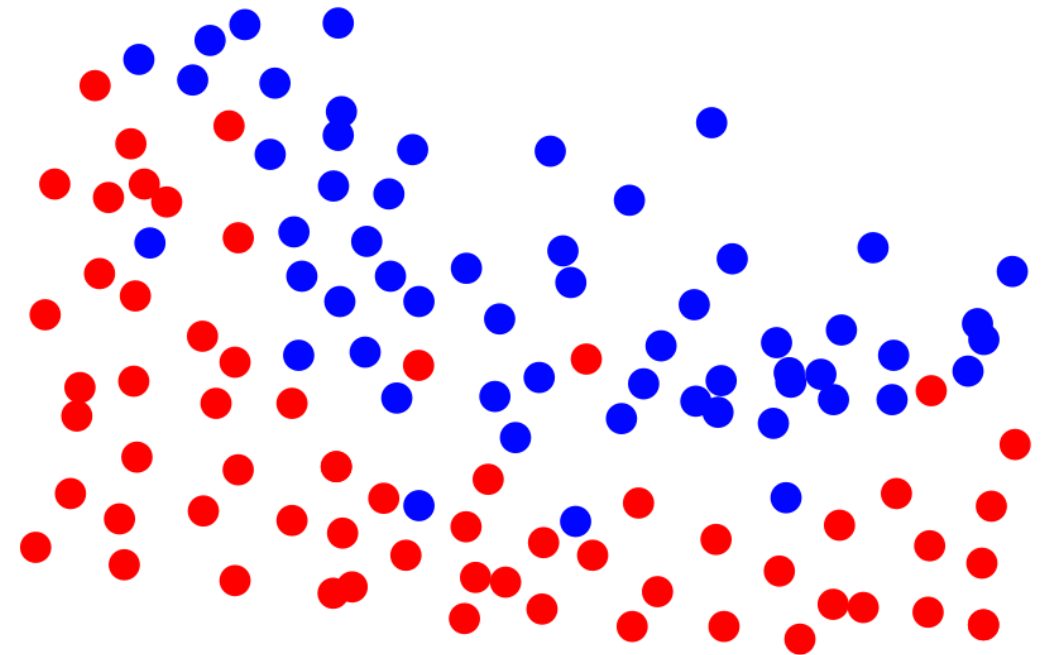
Table of Contents

- Introduction
- **Machine Learning**
- Logistic Regression

➤ The slides are based on https://evol.bio.lmu.de/_statgen/Rcourse/ws1617/

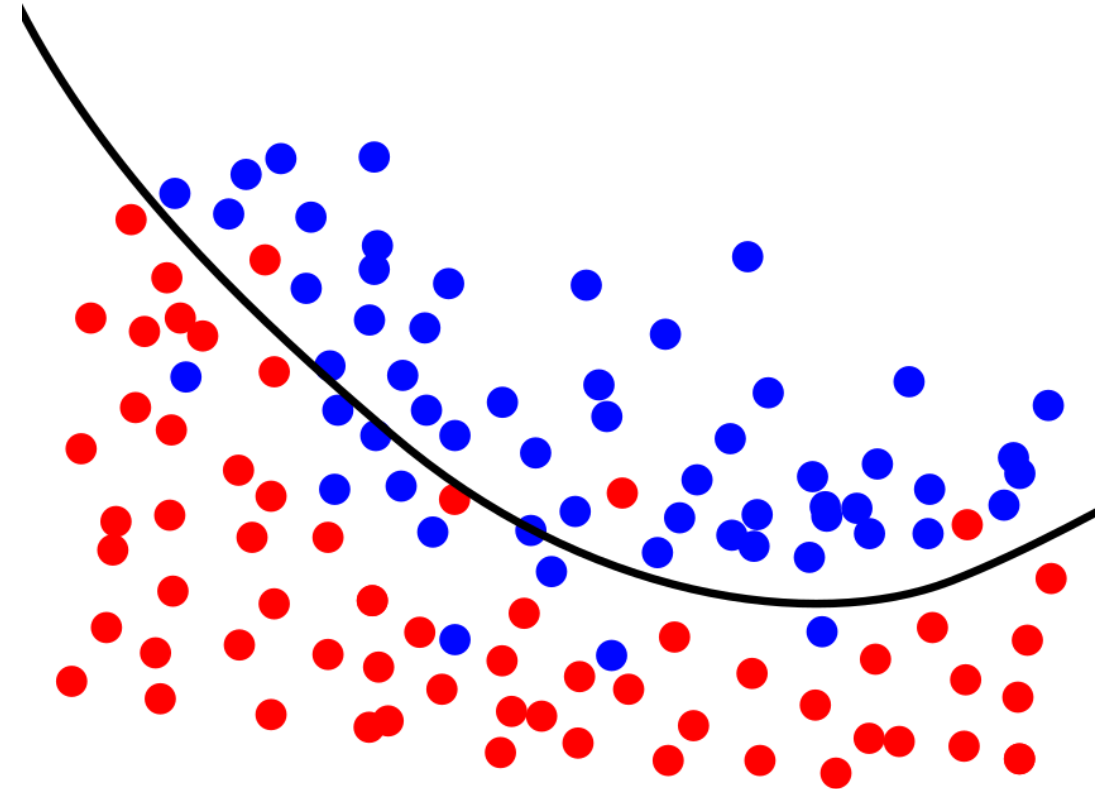
What is our goal?

- We have the dataset given
- Ultimately, we want to learn to separate the blue from the red points
- **How can we do that with a computer, such that it also learns with experience?**



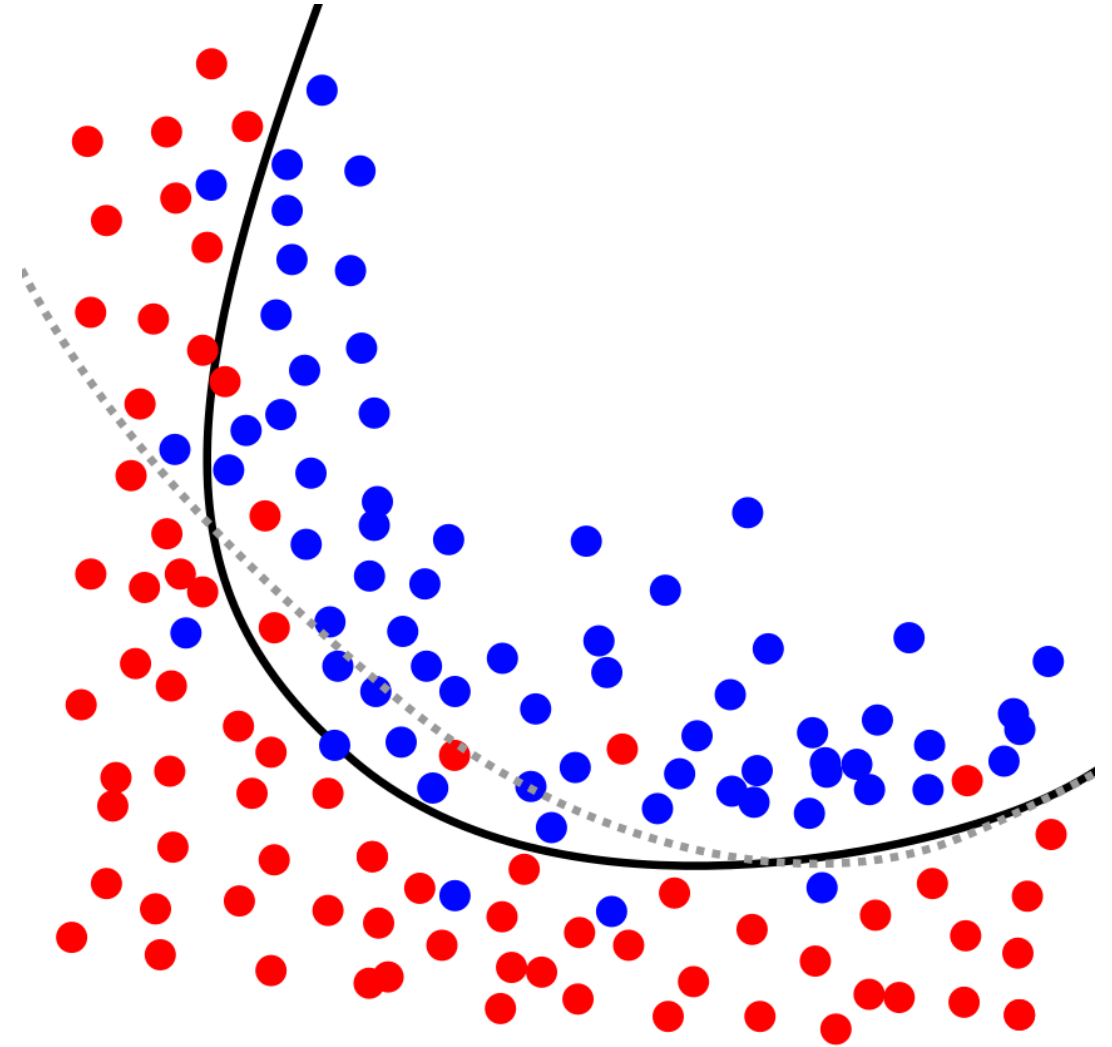
What is our goal?

- **Let's look at the following idea:**
- Instead of defining some complex rules or other complicated approaches, we enable the computer to:
 - Represent a certain function
 - Place the function in space such, that it optimally separates the blue from the red dots
- => The trained model is the definition of the separating line
- **Cool, but how does this improve with more data?**



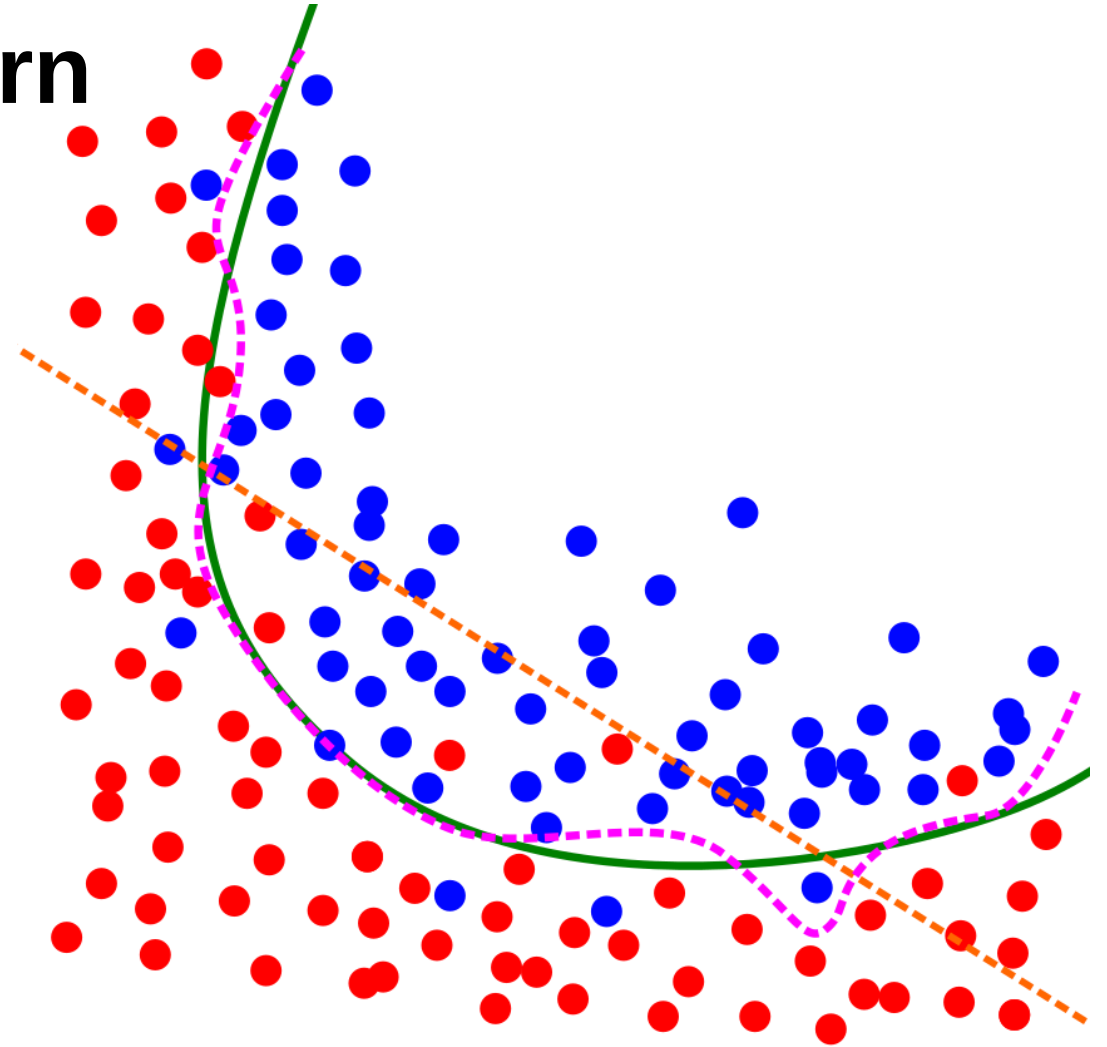
What is our goal?

- Simple, with every data point, we adjust the function such that the separation is still optimal.
- The more data we have, the better we can define the separating function
- This is exactly the definition of learning 😊
- **But how do we learn and store such a function**



How does the computer learn

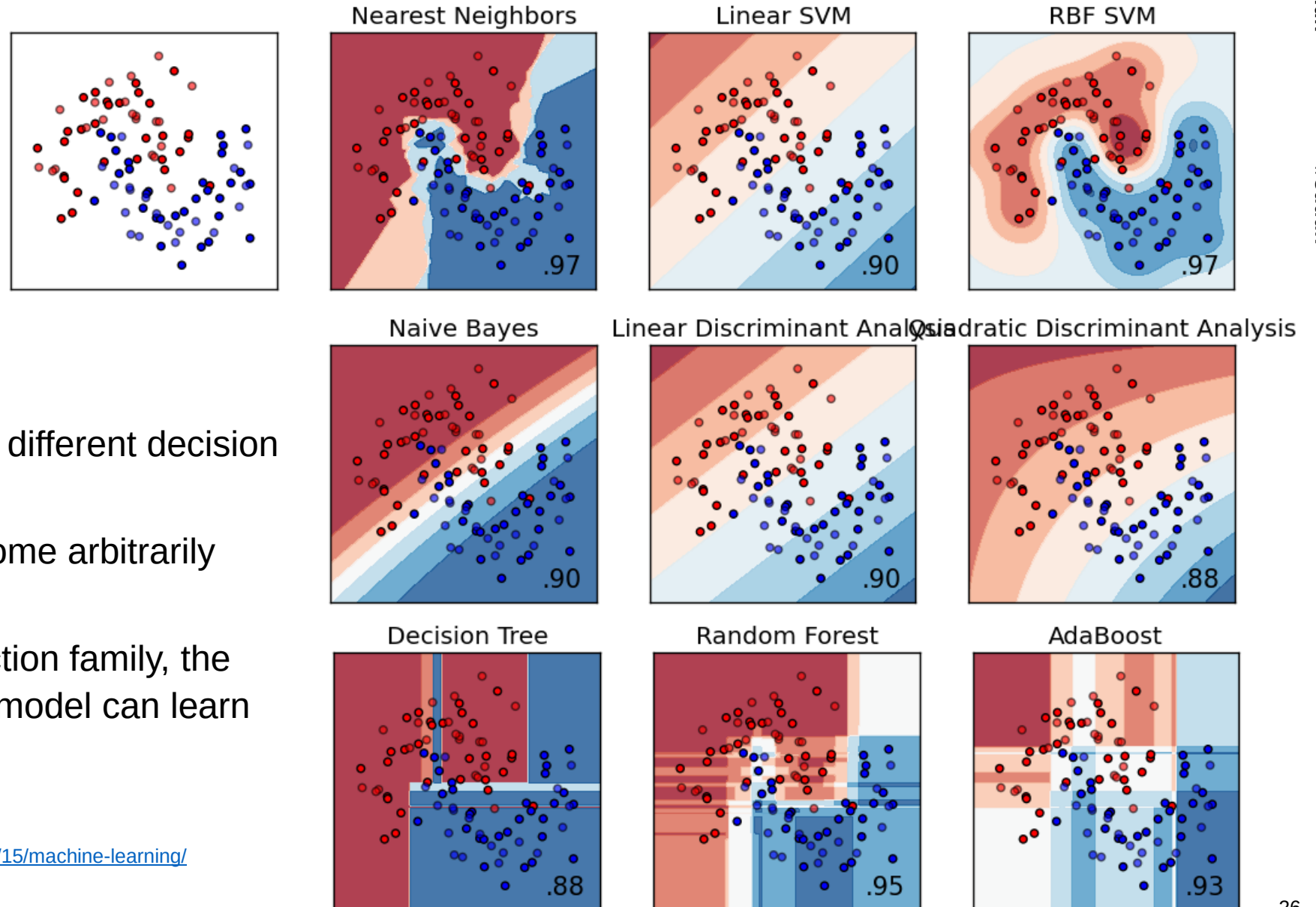
- **How can a computer learn such a function?**
- Main Concept
 - Define a family of functions which can separate our data
 - Each function is parametrized
 - Remember our linear regression, we had the slope and intercept
 - Finding the parameters is learning
- The different ML tools are only different in what function family they support and how they find this optimum
 - Linear Models
 - Polynomial Model
 - Neural Network
- **Is it really that simple?**



Yes, it is

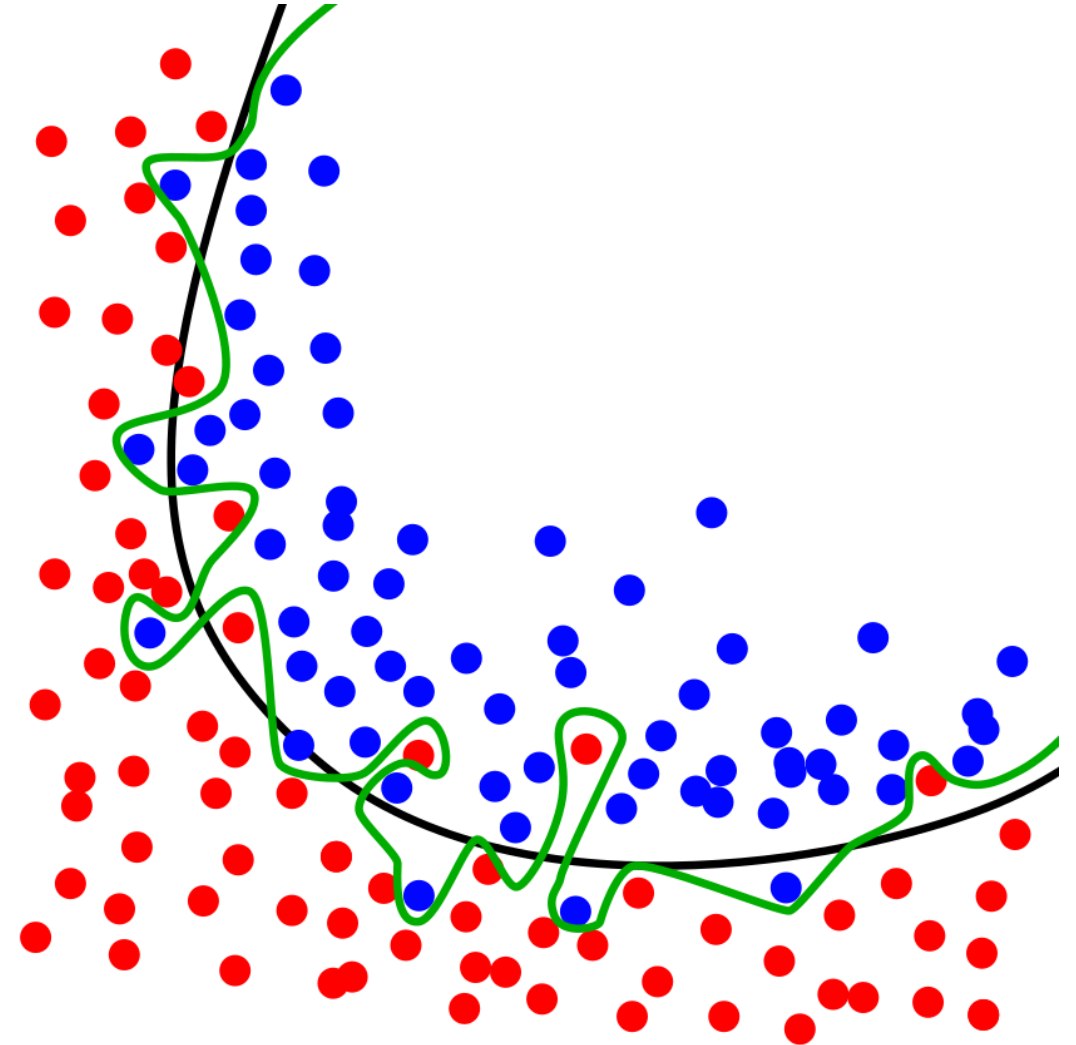
- Different ML tools just have different decision boundaries
- Those boundaries can become arbitrarily complex
- The more powerful the function family, the more complex task the ML model can learn

➤ Image adjusted from <https://shibberu.com/2016/01/15/machine-learning/>



What model to choose?

- Why are we not choosing the most complex model all the time?
- **Disadvantage of complex models**
 - Hard to train
 - Hard to interpret
 - Prone to overfitting
 - That is, we learn the data by heart
 - We perform poorly on validation data (i.e., data we have not used for training)
- **Disadvantages of simple models**
 - Poor performance
 - Often underfit
 - The model cannot grasp the complexity of the task



How to detect overfitting?

- Normally, we split our dataset into two parts
 - Training, validation and testing portion
 - Typical split: 70 / 20 / 10
- **Procedure**
 1. We train with the training data only
 2. We use the validation data to evaluate different model complexities
 3. We chose the model which gives the best validation performance
 4. Once the best model complexity is found, we train our final model on training and validation and evaluate on last time on the test data

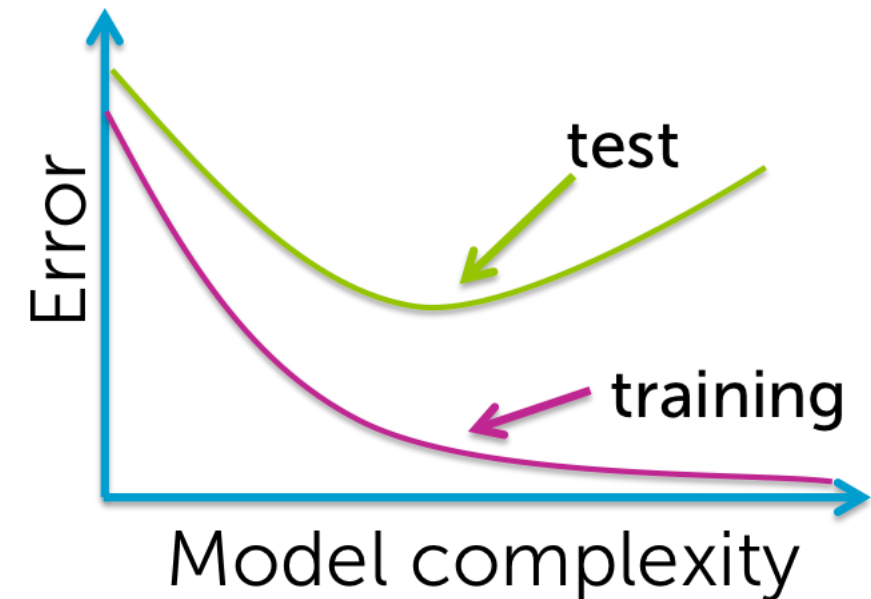
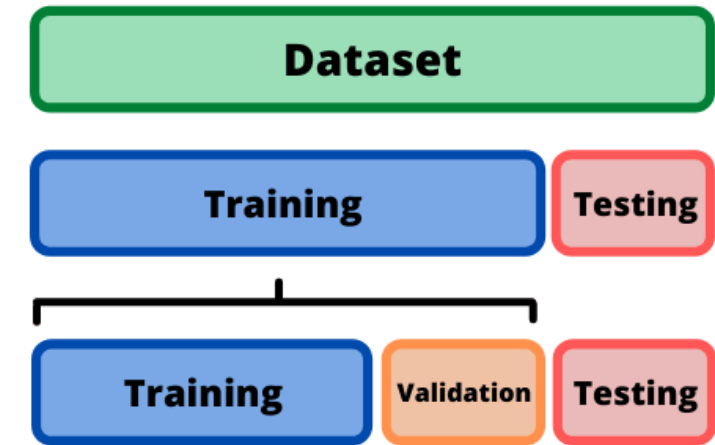
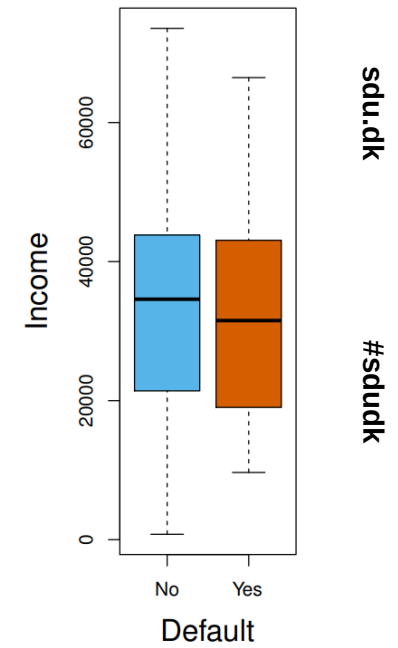
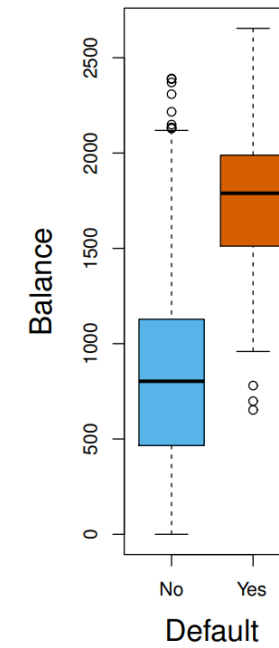
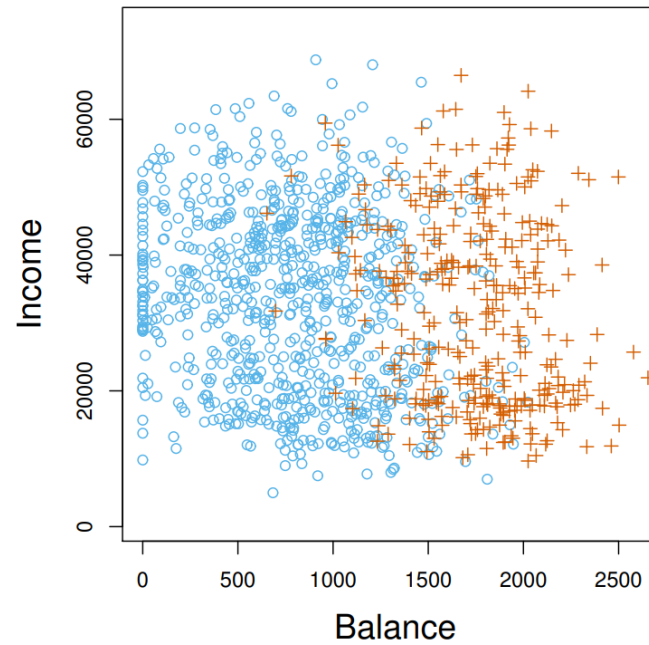


Table of Contents

- Introduction
- Machine Learning
- **Logistic Regression**

➤ The slides are based on https://evol.bio.lmu.de/_statgen/Rcourse/ws1617/

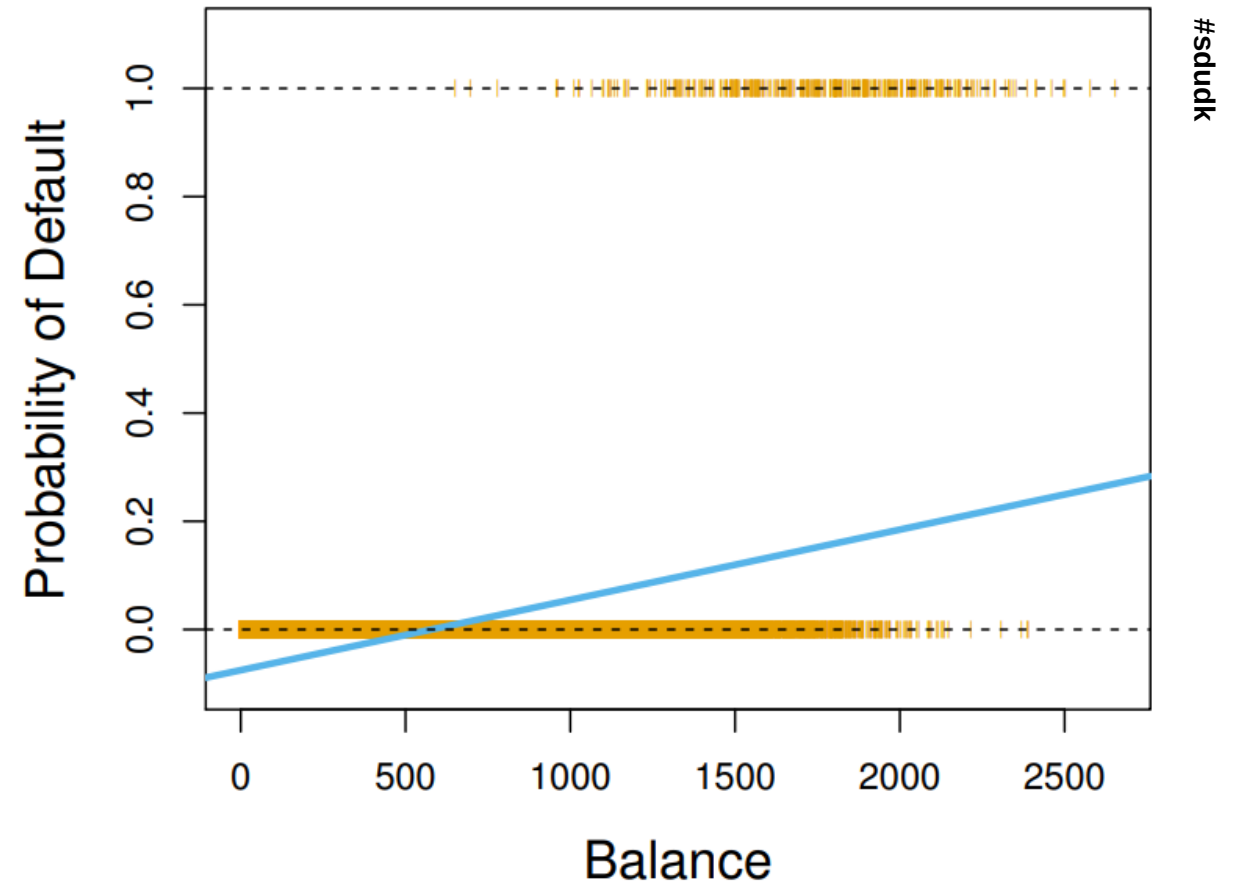
Logistic Regression



- Okay, we have understood machine learning in general
- Let's look at a concrete example:
 - The data indicates whether a credit is paid back
 - Two main variables:
 - Balance (that means how much we owe the bank)
 - Income (monthly income)
- Interestingly, it looks like the probability for defaulting is only depending on the balance
- Let's start with what we know

How about Linear regression?

- We can fit a linear regression
- We can define, once our regression value exceeds 0.5, we predict one class, otherwise we predict the other class
- **Problems with this approach:**
 - predicts proportions outside $[0, 1]$
 - assumes variance homogeneity (same precision for all observations)
 - proportions not normally distributed.



Bernoulli Modelling of the Default Data

- Lets consider a Bernoulli experiment determining whether a customer X_i defaults

$$P(X_i = 1) = p_i$$

$$P(X_i = 0) = 1 - p_i$$

- The likelihood of a customer defaulting is defined by p_i
- This variable is basically defining the bias of the coin
- This value should be somehow dependent on the customer information, i.e., data
- There are many ways of modelling this, any function would do.
 - Most simple version: linear regression

$$p_i = \alpha + \beta \text{balance}_i$$

- Main problem remains: Now p_i is not bounded



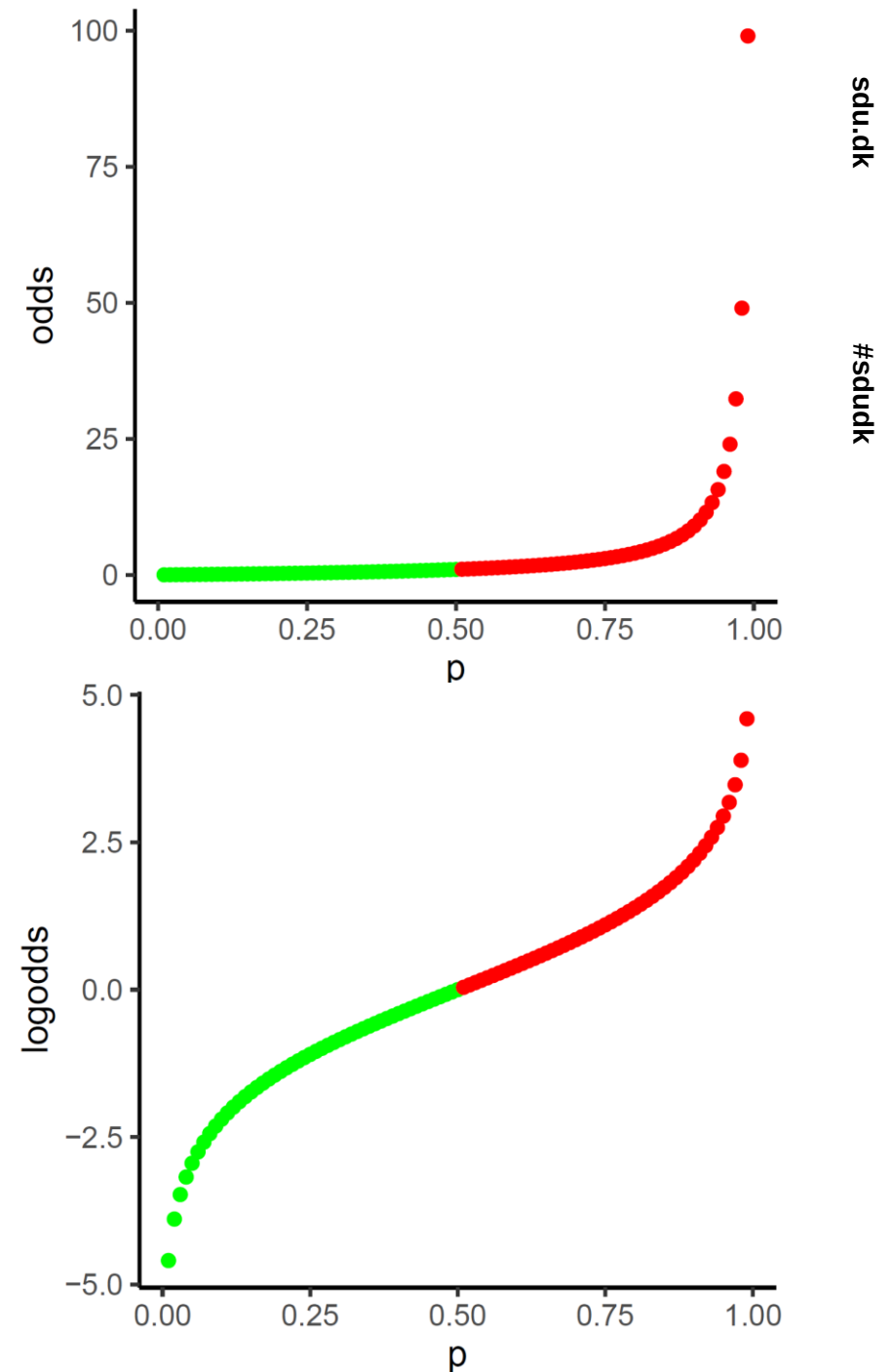
Introduction to the Log-Odds

- Maybe it helps to model something else, a quantity which is unbounded
- Betting odds:

$$\text{odds} = \frac{p_i}{1 - p_i}$$

- That means, if defaulting is twice as likely as paying, we receive a value of 2, if paying is twice as likely, then we receive a values of 0.5
- Still not ideal, the value is bound to $]0, \infty[$.
- **Solution: log-transform**

$$\text{logodds} = \log \frac{p_i}{1 - p_i}$$



The Logistic Regression

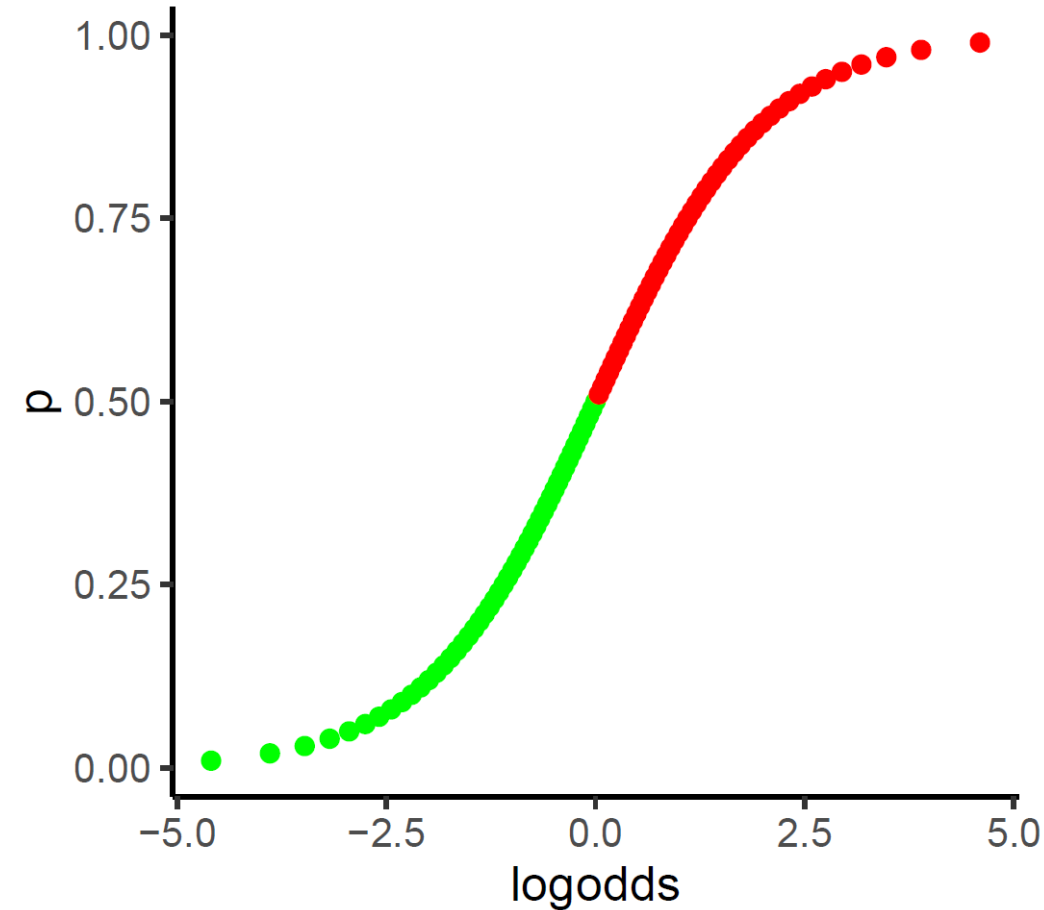
- We now can model the log-odds, which are unbounded with our linear function

$$\eta = \log \frac{1}{1 - p_i} = \alpha + \beta x$$

- A brief transformation leads to

$$\eta = \alpha + \beta x \quad \Leftrightarrow \quad p = \frac{\exp(\alpha + \beta x)}{\exp(\alpha + \beta x) + 1}$$

- Now the values are bound between $]0,1[$ and we have a clear interpretation of our model



Logistic Regression in R

```
> glm.fits <- glm(default ~ balance, data = credit_data, family = binomial)
> summary(glm.fits)
```

Call:

```
glm(formula = default ~ balance, family = binomial, data = credit_data)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.2697	-0.1465	-0.0589	-0.0221	3.7589

Coefficients:

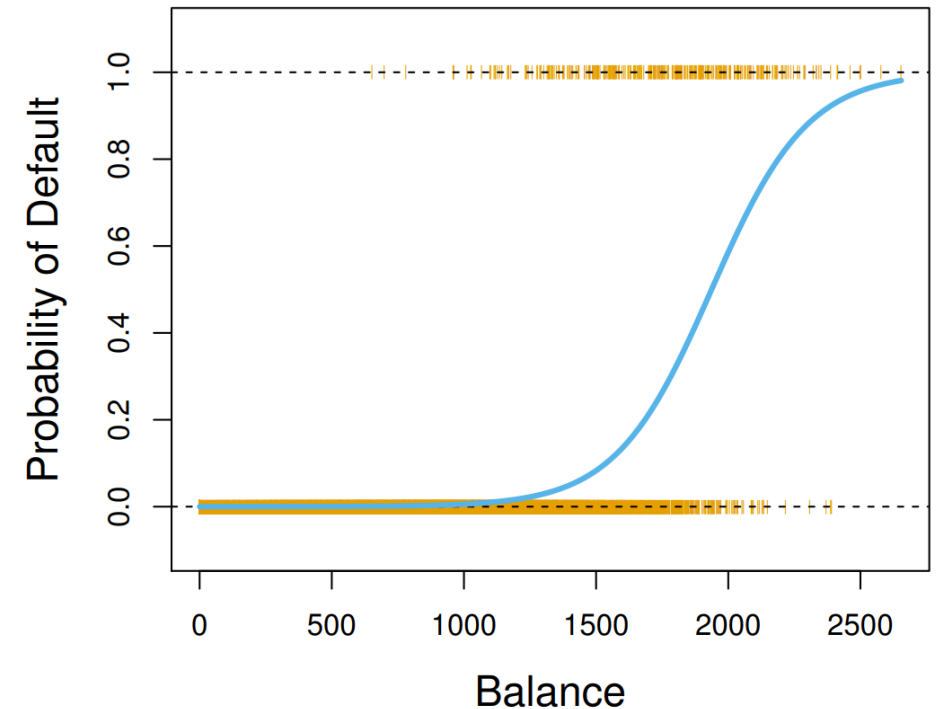
	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.065e+01	3.612e-01	-29.49	<2e-16 ***
balance	5.499e-03	2.204e-04	24.95	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 2920.6 on 9999 degrees of freedom
Residual deviance: 1596.5 on 9998 degrees of freedom
AIC: 1600.5

Number of Fisher Scoring iterations: 8



Logistic Regression in R

- In order to perform a logistic regression, we use the package `glm`
- The format is mostly the same as for linear regression:
 - The formular part describes the regression formula
 - New is the family: Here we specify we want to make a binomial model, i.e., a logistic regression
 - `glm` stands for generalized linear models, so they can do more than only logistic regression
- How to interpret the coefficients:
 - The same way as with linear regression
 - One unit increase in **balance** results in 0.0055 increase in the logodds for defaulting

```
> glm.fits

Call:  glm(formula = default ~ balance, family = binomial, data = credit_data)

Coefficients:
(Intercept)      balance 
 -10.651331      0.005499 

Degrees of Freedom: 9999 Total (i.e. Null);  9998 Residual
Null Deviance:      2921 
Residual Deviance: 1596      AIC: 1600
```

Multiple Linear Regression

- The dataset contains more variables
 - Balance
 - Income
 - Student (yes/no)
- We can construct the same multiple regressions as with the simple linear regression
- The formula is straight forward

$$p = \frac{\exp(\alpha + \beta_1 x_1 + \beta_2 x_2 + \dots)}{\exp(\alpha + \beta_1 x_1 + \beta_2 x_2 + \dots) + 1}$$

```
> glm.fits <- glm(default ~ ., data = credit_data, family = binomial)
> summary(glm.fits)

Call:
glm(formula = default ~ ., family = binomial, data = credit_data)

Deviance Residuals:
    Min       1Q   Median       3Q      Max 
-2.4691  -0.1418  -0.0557  -0.0203   3.7383 

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.087e+01  4.923e-01 -22.080  < 2e-16 ***
studentYes   -6.468e-01  2.363e-01  -2.738  0.00619 **
balance       5.737e-03  2.319e-04  24.738  < 2e-16 ***
income        3.033e-06  8.203e-06   0.370  0.71152
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

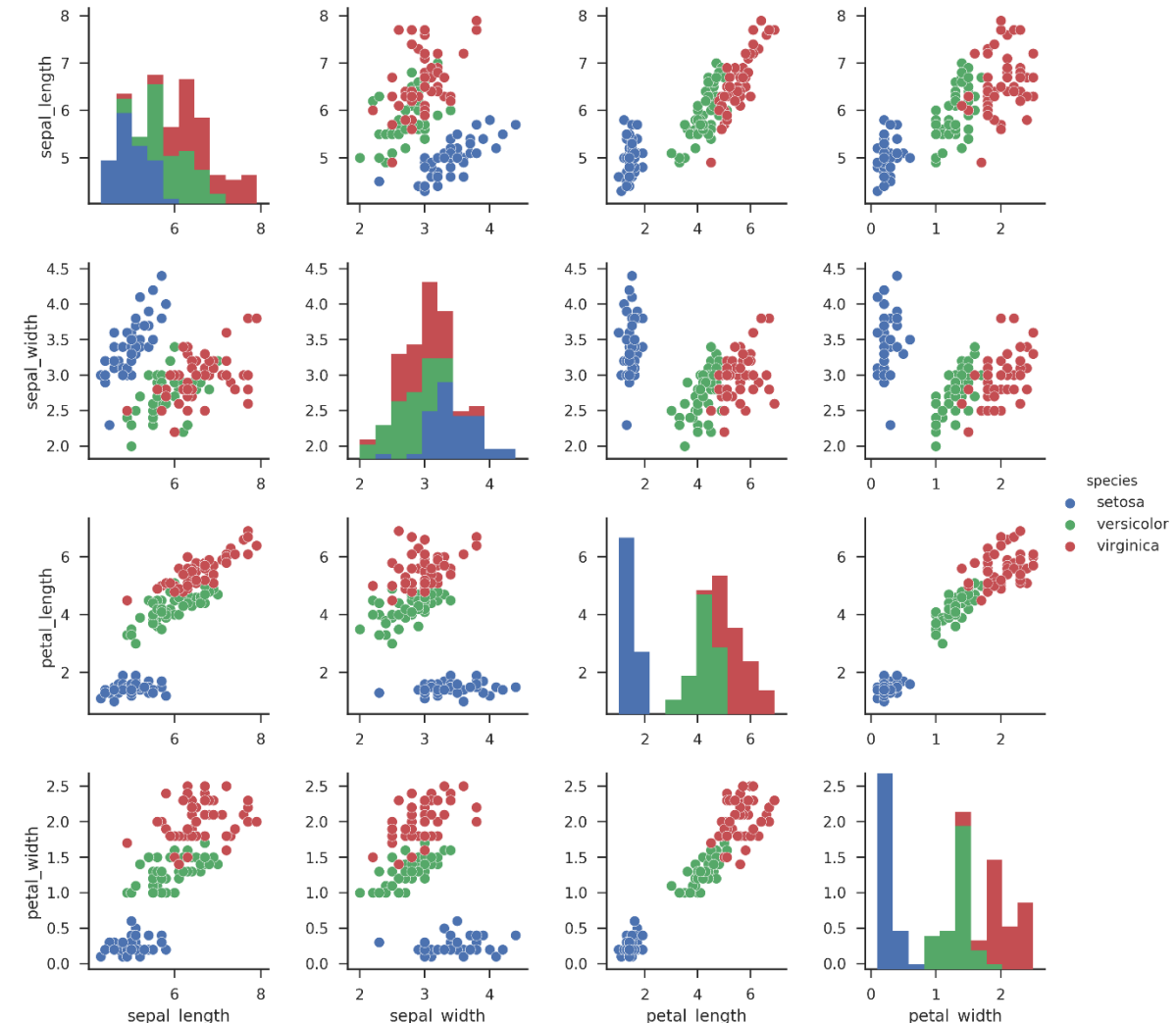
(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2920.6  on 9999  degrees of freedom
Residual deviance: 1571.5  on 9996  degrees of freedom
AIC: 1579.5

Number of Fisher Scoring iterations: 8
```

Limitations of a linear Regression

- It models the logodds in a linear fashion
- Most simplistic model assumption
- The outcome variable must be binary
- **What about multiple outcomes?**
 - Without going into details, it can be solved as multiple one-vs-rest logistic regressions
 - In R, there exist a package which does this for us



Using Multinomial Logistic regression

- We have three classes (*setosa*, *versicolor*, and *virginica*), but we only see two blocks of coefficients
- nnet compares vs the baseline assumption of being *setosa*:

$$\log\left(\frac{P(\text{Specis} = \text{versicolor})}{P(\text{Specis} = \text{setosa})}\right) = 18.69 - 5.46 \cdot \text{Sepal.Length} - 8.71 \cdot \text{Sepal.Width} + 14.24 \cdot \text{Petal.Length} - 3.10 \cdot \text{Petal.Width}$$
$$\log\left(\frac{P(\text{Specis} = \text{versicolor})}{P(\text{Specis} = \text{setosa})}\right) = -23.84 - 7.92 \cdot \text{Sepal.Length} - 15.37 \cdot \text{Sepal.Width} + 23.66 \cdot \text{Petal.Length} + 15.14 \cdot \text{Petal.Width}$$

```
> require(nnet)
> model <- nnet::multinom(Species ~., data = iris)
> summary(model)
Call:
nnet::multinom(formula = Species ~ ., data = iris)

Coefficients:
              (Intercept) Sepal.Length Sepal.Width Petal.Length Petal.Width
versicolor      18.69037      -5.458424      -8.707401      14.24477      -3.097684
virginica       -23.83628      -7.923634     -15.370769      23.65978      15.135301

Std. Errors:
              (Intercept) Sepal.Length Sepal.Width Petal.Length Petal.Width
versicolor      34.97116      89.89215      157.0415      60.19170      45.48852
virginica       35.76649      89.91153      157.1196      60.46753      45.93406

Residual Deviance: 11.89973
AIC: 31.89973
```